

# **B.Sc. (Data Science)**

## **DS101T : Problem Solving and Python Programming**

### **Unit 4: Modules and Packages**

**Mrs. Reshma Masurekar**  
**Dr. D. Y. Patil Arts, Commerce and Science College,**  
**Pimpri, Pune 18**

# Modules and Packages

- 3.1 Built in Modules
  - 3.1.1 Importing modules in python program
  - 3.1.2 Working with Random Modules.
  - 3.1.3 E.g. - built-ins, time, date time, calendar, sys, etc
- 3.2 User Defined functions
  - 3.2.1 Structure of Python Modules
- 3.3 Packages
  - 3.3.1 Predefined Packages
  - 3.3.2 User defined Packages

# Modules

- A modules are (.py)files which contain Python code defining functions, classes, variables and statements to perform a specific task.
- A file containg .py python code is called as module.
- Ex. test.py, is called a module, and its name is test.
- Python provides various types of modules:
  - built-in python modules
  - User-defined python modules
- Python supports tons of standard modules.

# Importing modules in python program

- To use the functionality present in any module, it needs to import in current program.
- ***import keyword*** along with the desired module name is used .
- When the interpreter comes across an import statement, it imports the module to your current program.
- Functions inside a module are used by using a dot (.) operator along with the module name.

- Example:

```
import math
```

```
n=4
```

```
math.sqrt(4)
```

- Import the definitions inside a module:

```
Import p1
```

```
print(p1.add(10,20))
```

- Import the definitions using interactive interpreter:

```
>>> import p1
```

```
>>> print(p1.add(10,20))
```

- Importing objects from module:

- a. **from x import a**

- Imports module x, and create the references in the current namespace to all public objects defined by that module. If we run this statement, we can simply use a plain name to refer to things defined in module x.
    - Attributes or method can be accessed directly without dot notation.
    - Import the definitions inside a module:  
from p1 import add  
print(add(10,20))
    - For import on interactive interpreter:  
>>> from p1 import add  
>>> add(10,20)

## **b. from x import a,b,c**

- Imports module x, and create the references in the current namespace to all public objects defined by that module.
- Or use a(),b(), c() are functions in program.
- Attributes or method can be accessed directly without dot notation.
- Import the definitions inside a module:

```
from p1 import add, sub
print(add(20,10))
print(sub(20,10))
```
- For import on interactive interpreter:

```
>>> from p1 import add, sub
>>> add(20,10)
>>> sub(20,10)
```

### c. **from x import \***

- \* is used to import all functions and variables from the module.
- Import the definitions inside a module:

```
from p1 import *  
print(add(20,10))  
print(sub(20,10))
```
- For import on interactive interpreter:

```
>>> from p1 import *  
>>> add(20,10)  
>>> sub(20,10)
```

# Aliasing modules

- It is used to modify name of modules and their functions within Python by using 'as' keyword.
- Import module as another\_name
- Ex.: import p1 as m  
m.add(20,10)

# User Defined Modules:

- Creating a user-defined module in Python involves writing functions, classes, or variables in a Python file and then importing this file into other Python scripts.
- Create a new Python file with a .py extension

**mymodule.py**

```
def add(a, b):  
    return a + b
```

```
def subtract(a, b):  
    return a - b
```

```
PI = 3.14159
```

```
def greet(name):  
    return f"Hello, {name}!"
```

# # Importing the entire module

# main.py

import mymodule

|                                 |                          |
|---------------------------------|--------------------------|
| print(mymodule.add(5, 3))       | # Output: 8              |
| print(mymodule.subtract(5, 3))  | # Output: 2              |
| print(mymodule.PI)              | # Output: 3.14159        |
| print(mymodule.greet("AaKash")) | # Output: Hello, Aakash! |

## #importing Specific Items

# main.py

from mymodule import greet, PI

|                     |                       |
|---------------------|-----------------------|
| print(greet("Bob")) | # Output: Hello, Bob! |
| print(PI)           | # Output: 3.14159     |

## #Using Alias

import mymodule as mm

|                         |                        |
|-------------------------|------------------------|
| print(mm.greet("Arya")) | # Output: Hello, Arya! |
|-------------------------|------------------------|

# Python built-in modules

- Python **built-in modules** are a *set of libraries that come pre-installed with the Python installation*.
- These modules provide a wide range of functionalities, from file operations and system-related tasks to mathematical computations and web services.
- The use of these modules simplifies the development process, as developers can leverage the built-in functions and classes for common tasks, providing code reusability, and efficiency.
- Built-in modules include “os”, “sys”, “math”, and “datetime”.

- Numeric and mathematical modules:
  - a. math: logarithmic, trigonometric, hyperbolic functions of real numbers
  - b. cmath: mathematical functions of complex numbers
  - c. random
  - d. decimal
  - e. fraction
  - f. statistics
  - g. numbers

# math

- Provides access to mathematical functions.

| Functions                               | Use                                                 | Example                                                          |
|-----------------------------------------|-----------------------------------------------------|------------------------------------------------------------------|
| <a href="#"><u>math.sqrt()</u></a>      | Returns the square root of a number                 | math.sqrt(4)                                                     |
| <a href="#"><u>math.ceil()</u></a>      | Rounds a number up to the nearest integer           | math.ceil(1.001)=>2                                              |
| <a href="#"><u>math.floor()</u></a>     | Rounds a number down to the nearest integer         | math.floor(1.001)=>1                                             |
| math. <b>remainder</b> (x, y)           | remainder of x with respect to y                    | math.remainder(4,2)                                              |
| <a href="#"><u>math.factorial()</u></a> | Returns the factorial of a number                   | <a href="#"><u>math.factorial(3)</u></a>                         |
| <a href="#"><u>math.fabs()</u></a>      | Returns the absolute value of a number              | print(math.fabs(-66.43))<br>print(math.fabs(-7))<br>66.43<br>7.0 |
| <a href="#"><u>math.gcd()</u></a>       | Returns the greatest common divisor of two integers | <a href="#"><u>math.gcd()</u></a>                                |
| <a href="#"><u>math.pow()</u></a>       | Returns the value of x to the power of y            | <a href="#"><u>math.pow(3,2)</u></a>                             |
| <a href="#"><u>math.sin()</u></a>       | Returns the sine of a number                        | <a href="#"><u>math.sin()</u></a>                                |

# random

- TO pick random numbers in a given range.
- Import random
  - random():
  - randint(lower, upper)
  - randrange(lower, upper)

# statistics

- Import statistics
  - `statistics.mean([1,2,3])`
  - `statistics.median([1,2,3])`
  - `statistics.mode([1,2,3])`
  - `statistics.stdev([1,2,3])`

# sys

- **sys module** in Python provides various functions and variables that are used to manipulate different parts of the *Python runtime environment*.
- It allows operating on the interpreter as it provides access to the variables and functions that interact strongly with the interpreter.
- **sys.version**

## Variables:=>

|                             |                                                                                                                                                                       |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sys.version</code>    | returns a string containing the version of Python Interpreter with some additional information. This shows how the sys module interacts with the interpreter.         |
| <code>sys.path :</code>     | This function displays the current system's python path. It's an environmental variable that contains the search path for all of the <a href="#">python modules</a> . |
| <code>sys.executable</code> | A string containing the absolute path of the Python interpreter's executable binary.                                                                                  |
| <code>sys.argv</code>       | This includes a list of arguments (command prompt) supplied to the script.                                                                                            |

## Functions:=>

|                               |                                                                                                                                                                                  |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sys.exit():</code>      | The program will exit the python console or the command prompt. It is usually used to exit the application if an exception is thrown securely.                                   |
| <code>sys.getsizeof():</code> | After using this function, the size of an object in bytes is returned. Any object can be used as an object.<br>Example: <code>i=3</code><br><code>print(sys.getsizeof(i))</code> |

# OS

- os: Python has a built-in os module with methods for interacting with the operating system, like creating files
- and directories, management of files and directories, input, output, environment variables, process
- management, etc.

|                                                  |                                                                      |
|--------------------------------------------------|----------------------------------------------------------------------|
| <code>os.name()</code>                           | provides the name of the operating system module that it imports.    |
| <a href="#"><u><code>os.getcwd()</code></u></a>  | Returns the current working directory                                |
| <a href="#"><u><code>os.getegid()</code></u></a> | Return the effective group id of the current process                 |
| <a href="#"><u><code>os.abort()</code></u></a>   | Terminates a running process immediately                             |
| <b><code>os.mkdir()</code></b>                   | used to create new directory.Ex. <code>os.mkdir("d:\\newdir")</code> |
| <code>os.chdir()</code>                          | to change the current working directory.                             |
| <code>os.rmdir()</code>                          | removes the specified directory with an absolute or related path.    |
| <code>os.close()</code>                          | This function closes the associated file with descriptor             |

# datetime

- Supplies classes for manipulating dates and times.
- `import datetime`
- `x=datetime.datetime.now()` : display current date

`date.today()`

To return the current date(year, month, and day).  
`today = date.today()`

```
print("Current year:", today.year)
print("Current month:", today.month)
print("Current day:", today.day)
```

`weekday()`

Returns the day of the week as integer where Monday is 0 and Sunday is 6

`Time()`

`Time = time(11, 34, 56)`

```
print("hour =", Time.hour)
print("minute =", Time.minute)
print("second =", Time.second)
print("microsecond =", Time.microsecond)
```

*`Datetime()`*

information on both date and time.

`datetime.now()`

returns the current local date and time.

# package

- Package is a collection of python modules with an additional `__init__.py` file.
- Package is a hierarchical file data structure that defines a single python application environment that consist of modules and subpackages and so on.
- **Types of Packages:**
  1. Predefined Packages
  2. User defined Package



# Top Python libraries a Data Scientist need to know

01

Pandas

02

NumPy

03

SciPy

04

Scrapy

05

Matplotlib

Seaborn

06

Scikit-Learn

07

TensorFlow

08

Scikit-Image

09

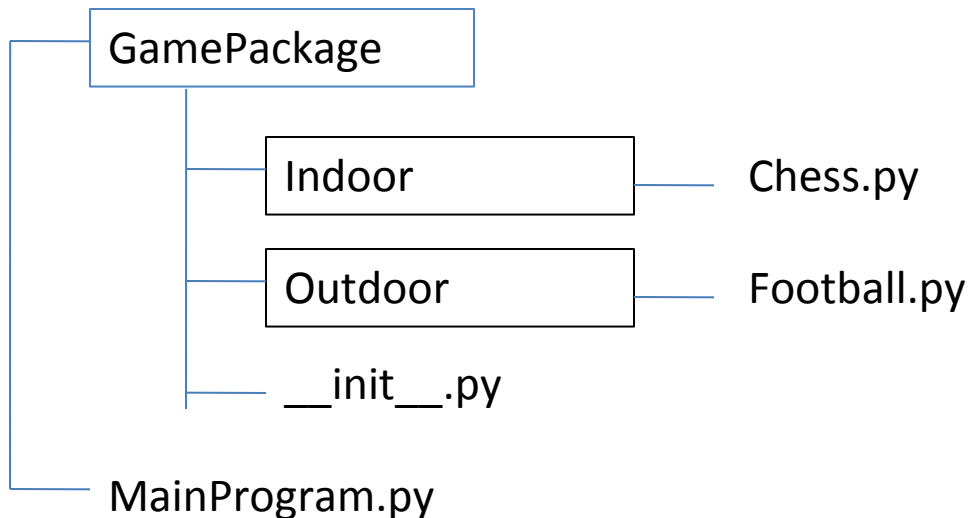
Librosa

10



# User defined package

- Creating and using user-defined packages in Python allows you to organize your code into modular, reusable components.
- **Example:**



# Step-by-Step process

**Step1:** Create a Directory Structure: First, create a directory structure for your package.

Example: we'll create a package named mypackage with two modules: math\_operations and string\_operations.

**Step2:** Create Modules:

Example: Create the modules math\_operations.py and string\_operations.py.

**Step3: Initialize the Package**

Create the `__init__.py` file in the mypackage directory to make it a package. This file can be empty or you can use it to import specific functions or classes to be available at the package level.

`__init__.py`

**Step4: Using the Package**

Create a main script that uses the mypackage package.

# User-defined Packages:

- Creating and using user-defined packages in Python allows you to organize your code into modular, reusable components.

## **Chess.py**

```
def Chessprint():  
    print("I like to play Chess")
```

## **Football .py**

```
def Footballprint():  
    print("I like to play Football")
```

## **\_\_init\_\_.py**

```
from GamePackage.Indoor.Chess import Chessprint  
from GamePackage.Outdoor.Football import Footballprint
```

## **MainProgram.py**

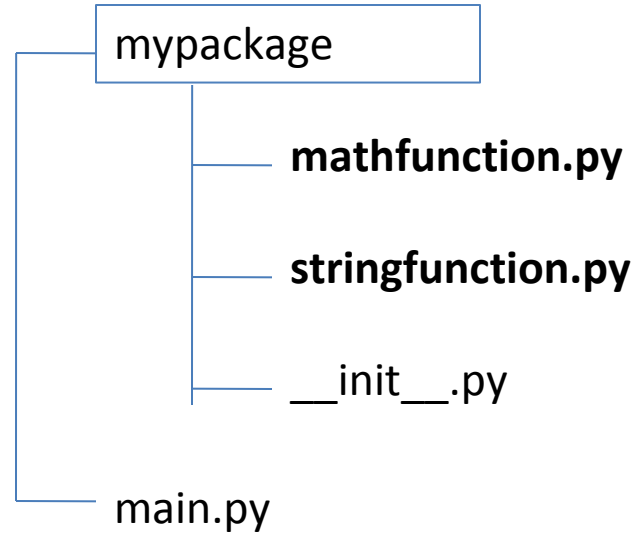
```
Import GamePackage  
Chessprint()  
Footballprint()
```

## **mathfunction.py**

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b  
  
def divide(a, b):  
    return a / b
```

## **\_\_init\_\_.py**

```
from mypackage.mathfunction import *  
from mypackage.stringfunction import *
```



## **stringfunction.py**

```
def to_uppercase(s):  
    return s.upper()  
  
def to_lowercase(s):  
    return s.lower()  
  
def reverse_string(s):  
    return s[::-1]
```

## **main.py**

```
import mypackage
```

```
# Using functions from math_operations module
```

```
print("Math Operations:")
```

```
print("Add:", mypackage.add(5, 3))
```

```
print("Subtract:", mypackage.subtract(5, 3))
```

```
print("Multiply:", mypackage.multiply(5, 3))
```

```
print("Divide:", mypackage.divide(5, 3))
```

```
# Using functions from string_operations module
```

```
print("\nString Operations:")
```

```
print("Uppercase:", mypackage.to_uppercase("hello"))
```

```
print("Lowercase:", mypackage.to_lowercase("HELLO"))
```

```
print("Reverse:", mypackage.reverse_string("hello"))
```

# Built in package

- **Predefined Packages:** The total number of Python packages in existence exceeds 200,000 (and that figure only includes those stored on PyPI, the official Python Package Index).
- Python packages offer a user-friendly and effective solution to tackle challenging issues in a variety of domains, including scientific computing, data visualisation, data modelling, and many more.
- It is better to complete complicated things piecemeal, one subtask at a time.
- To accomplish tasks, programmers build and use modules, which are collections of related code kept in independent files.

# NumPy

- NumPy stands for **numeric python** which is a python package for the *computation and processing of the multidimensional and single dimensional array elements*.
- NumPy provides various powerful data structures, implementing multi-dimensional arrays and matrices. These data structures are used for the optimal computations regarding arrays and matrices.
- NumPy provides a convenient and efficient way to handle the vast amount of data.
- NumPy is also very convenient with Matrix multiplication and data reshaping. NumPy is fast which makes it
- reasonable to work with a large set of data.

# Pandas

- Pandas is used for structured data operations and manipulations. Pandas provides various high-performance and easy-to-use data structures and operations for manipulating data in the form of numerical tables and time series.
- It provides easy data manipulation, data aggregation, reading, and writing the data as well as data visualization.
- Pandas can also take in data from different types of files such as CSV, excel etc.or a SQL database and create a Python object known as a data frame.
- The two main data structures that Pandas supports are:
  - Series: Series are one-dimensional data structures that are a collection of any data type.
  - DataFrames: DataFrames are two-dimensional data structures which resemble a database table or say an Excel spreadsheet

# Matplotlib:

- The most popular data exploration and visualisation library is Matplotlib.
- It can be used to create simple graphs such as linear plots, histograms, scatter graph, bar graphs, and pie charts.
- With the help of this library, you can also produce animated and interactive visualisations.
- Matplotlib is the foundation of every other visualisation library.